IMPERIAL

Department of Mathematics
Imperial College London

Formalizing Group Action in Lean

Student name: *Frankie Feng-Cheng WANG*
Email: *maths@frankie.wang*
GitHub: *https://github.com/FrankieeW/GroupAction*

Course: *MATH70040-Formalising Mathematics* – Lecturer: *Dr Bhavik Mehta*
Due date: *January 29, 2026*

Contents

1. Introduction

This report presents a Lean 4 formalisation of **group actions**, organised around three interrelated themes:

1. **Core definitions:** the `GroupAction` typeclass, together with the notions of faithfulness and transitivity;
2. **Concrete examples:** a selection of standard group actions illustrating the abstract definitions;
3. **Key theorems:** the construction of the permutation representation and the subgroup structure of stabilisers.

The mathematical development and theorem numbering follow Fraleigh and Katz, *A First Course in Abstract Algebra*, Addison–Wesley, 2003, Section 16 (Group Actions).
The complete Lean source code is available on GitHub:
`https://github.com/FrankieeW/GroupAction`.
All code snippets in this report are hyperlinked to the corresponding lines in the repository.
The principal results formalised in this project are the existence of the permutation representation associated to a group action and the fact that stabilisers form subgroups. These are stated below and proved in later sections.

> **Theorem 16.3.** Let X be a G-set. For each $g \in G$, the map
>
> $$\sigma_g : X \to X, \qquad \sigma_g(x) = g \cdot x,$$
>
> is a permutation of X. The assignment $\phi : G \to \mathrm{Sym}(X)$ defined by $\phi(g) = \sigma_g$ is a group homomorphism, and for all $g \in G$ and $x \in X$,
>
> $$\phi(g)(x) = g \cdot x.$$

> **Theorem 16.12.** Let X be a G-set and let $x \in X$. The stabiliser of x in G, defined by
>
> $$G_x = \{\, g \in G \mid g \cdot x = x \,\},$$
>
> is a subgroup of G.

2. Definitions

This section introduces the group action class and the standing assumptions used throughout the file.

A **group action** of a group G on a set X is a map

$$\cdot : G \times X \to X,$$

satisfying the following axioms for all $g_1, g_2 \in G$ and $x \in X$:

- (Compatibility) $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x)$;
- (Identity) $1 \cdot x = x$.

2.1. Group Action. I first declare a minimal class for group actions:

```
class GroupAction (G : Type*) [Monoid G] (X : Type*) where
  act : G → X → X
  ga_mul : ∀ g₁ g₂ x, act (g₁ * g₂) x = act g₁ (act g₂ x)
  ga_one : ∀ x, act 1 x = x
```

Throughout the file I work with a group G acting on a type X:

```
variable {G : Type*} [Group G] {X : Type*} [GroupAction G X]
```

3. Concrete Examples of Group Actions

This section presents several concrete instances of the `GroupAction` class, illustrating how the abstract definition applies to familiar mathematical structures.

3.1. Symmetric group acting on a set. Let X be any type. The symmetric group $\mathrm{Sym}(X)$ acts on X by evaluation: for $\sigma \in \mathrm{Sym}(X)$ and $x \in X$, define $\sigma \cdot x := \sigma(x)$.

```
instance permGroupAction (X : Type*) : GroupAction (Equiv.Perm X) X :=
  { act := fun g x => g x
    ga_mul := by
      intro g1 g2 x
      rfl
    ga_one := by
      intro x
      rfl }
```

The axiom proofs are immediate by reflexivity: composition of permutations and function evaluation commute definitionally in Lean.
Two standard properties of this action are faithful and transitive:

```
theorem permGroupAction_faithful (X : Type*) :
  GroupAction.faithful (G := Equiv.Perm X) (X := X) :=
by
  intro g₁ g₂ h
  ext x
  exact h x

theorem permGroupAction_transitive
  (X : Type*) [DecidableEq X] :
  GroupAction.transitive (G := Equiv.Perm X) (X := X) :=
by
  intro x₁ x₂
  refine ⟨Equiv.swap x₁ x₂, ?_⟩
  simp [GroupAction.act]
--rmk: `[DecidableEq X]` is needed for `Equiv.swap`
```

The instance `DecidableEq X` is required because `Equiv.swap` constructs a permutation by branching on whether two elements are equal, which needs decidable equality.

3.2. Group acting on itself by left multiplication. Every group G acts on itself by left multiplication: $g_1 \cdot g_2 := g_1 * g_2$.

```
44 instance groupAsGSet (G : Type*) [Group G] : GroupAction G G :=
45   { act := fun g1 g2 => g1 * g2
46     ga_mul := by
47       intro g1 g2 g3
48       rw [mul_assoc]
49     ga_one := by
50       intro g
51       rw [one_mul] }
```

The `ga_mul` axiom follows from associativity of group multiplication, and `ga_one` from the identity axiom.

3.3. Subgroup acting on the group. A subgroup $H \leq G$ acts on G by left multiplication. Elements of H are coerced to elements of G using `(h : G)`.

```
55 instance subgroupAsGSet {G : Type*} [Group G] (H : Subgroup G) :
   ↪ GroupAction H G :=
56   { act := fun h g => (h : G) * g
57     ga_mul := by
58       intros
59       -- simp
60       -- rw [mul_assoc]
61       simp [mul_assoc]
62     ga_one := by
63       intros
64       simp }
```

The `simp` tactic handles the coercion and applies associativity and identity axioms.

3.4. Conjugation action of a subgroup on itself. A subgroup H acts on itself by conjugation: $h_1 \cdot h_2 := h_1 h_2 h_1^{-1}$.

```
66 instance subgroupAsGSetConjugation {G : Type*} [Group G] (H : Subgroup G)
   ↪ : GroupAction H H :=
67   {
68     act := fun h g => h * g * h⁻¹
69
70     ga_mul := by
71       intro g₁ g₂ g₃
72       -- one step
73       -- simp [mul_assoc]
74       -- in detail
75       simp
76       -- goal state: g₁ * g₂ * g₃ * (g₂⁻¹ * g₁⁻¹) = g₁ * (g₂ * g₃ * g₂⁻¹) *
          ↪ g₁⁻¹
77       rw [← mul_assoc g₁]
78       -- goal state: g₁ * g₂ * g₃ * (g₂⁻¹ * g₁⁻¹) = g₁ * (g₂ * g₃) * g₂⁻¹ *
          ↪ g₁⁻¹
79       rw [mul_assoc g₁ g₂]
```

```
80        -- goal state:  g₁ * (g₂ * g₃) * (g₂⁻¹ * g₁⁻¹) = g₁ * (g₂ * g₃) *
          ↪  g₂⁻¹ * g₁⁻¹
81        rw [← mul_assoc]
82      ga_one := by
83        intros
84        simp }
```

The `mul_assoc` lemma is from mathlib and states associativity of multiplication:

```
1  mul_assoc.{u_1} {G : Type u_1} [Semigroup G] (a b c : G) :
2  a * b * c = a * (b * c)
```

The `ga_mul` proof can easily be done by `simp [mul_assoc]`, but I expanded it to show the steps explicitly by using `rw`.
For example the `rw [← mul_assoc g₁]` will aotumatically find the passible situation looks like `g₁* (b * c)`, because we only give one argument `g₁` to `← mul_assoc`. In detail, it will rewrite the left hand side `g₁ * (g₂ * g₃ * g₂⁻¹)` to `g₁ * (g₂ * g₃) * g₂⁻¹` according to the right to left direction of the lemma `mul_assoc`.

3.5. Scalar action on complex vector spaces. The multiplicative group $\mathbb{C}^\times$ of nonzero complex numbers acts on $\mathbb{C}^n$ by scalar multiplication.

```
88  instance vectorSpaceAsCStarSet (n : ℕ) :
89      GroupAction (ℂˣ) (Fin n → ℂ) :=
90    { act := fun r v => fun i => (r : ℂ) * v i
91      ga_mul := by
92        intros r1 r2 v
93        ext i
94        simp [mul_assoc]
95      ga_one := by
96        intro v
97        ext i
98        simp }
```

Here `ℂˣ` denotes the units of $\mathbb{C}$ (invertible elements), and `Fin n → ℂ` represents functions from $\{0, \ldots, n-1\}$ to $\mathbb{C}$ (i.e., n-dimensional complex vectors). The `ext` tactic proves function equality pointwise.

3.6. Scalar action via units. The unit group $\alpha^\times$ of a monoid α acts on α^n by scalar multiplication.

```
103 instance vectorSpaceAsUnitSet
104   (α : Type*) [Monoid α]
105   (n : ℕ) :
106   GroupAction (αˣ) (Fin n → α) :=
107 { act := fun r v => fun i => (r : α) * v i
108   ga_mul := by
109     intros r1 r2 v
110     ext i
111     simp [mul_assoc]
112   ga_one := by
```

```
    intro v
    ext i
    simp }
```

This generalizes the complex example by abstracting over the coefficient monoid.

3.7. Dihedral group action on a square. Let D_4 be the symmetry group of a square, acting on $\mathbb{Z}/4\mathbb{Z}$.

```
abbrev D4 := DihedralGroup 4
abbrev Vertex := ZMod 4
abbrev Side := ZMod 4
abbrev Midpoint := ZMod 4

def d4Act (g : D4) (x : ZMod 4) : ZMod 4 :=
  match g with
  -- Rotation r_i: x ↦ x - i
  | DihedralGroup.r i => x - i
  -- Reflection s_i: x ↦ i - x
  | DihedralGroup.sr i => i - x

instance d4ActionZMod4 : GroupAction D4 (ZMod 4) :=
  { act := d4Act
    ga_mul := by
      intro g1 g2 x
      cases g1 <;> cases g2 <;>
        simp [d4Act, sub_eq_add_neg] <;> ring
    ga_one := by
      intro x
      simp [DihedralGroup.one_def, d4Act, -DihedralGroup.r_zero]
    }
```

4. Permutation Representation

The core construction is the map $\sigma_g : X \to X$ given by $\sigma_g(x) = g \cdot x$. The inverse of σ_g is $\sigma_{g^{-1}}$, so σ_g is a permutation. This yields the permutation representation $\phi : G \to \mathrm{Sym}(X)$.

4.1. Proof sketch (informal). To show that σ_g is a bijection, compute $\sigma_{g^{-1}}(\sigma_g(x)) = (g^{-1}g) \cdot x = 1 \cdot x = x$, and similarly $\sigma_g(\sigma_{g^{-1}}(x)) = x$. The representation is defined by $\phi(g) = \sigma_g$, and multiplicativity follows from the action axiom:

$$\phi(g_1 g_2)(x) = (g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = (\phi(g_1)\phi(g_2))(x).$$

4.2. Lean code. First, define the underlying action map:

```
/-- Defines the action map `σ_g : X → X` by `x ↦ g • x`. -/
def sigma (g : G) : X → X :=
  fun x => GroupAction.act g x
#check sigma
```

Then package it as a permutation using the inverse action:

```
/-- Defines the permutation of `X` induced by `g`.
    The inverse is given by the action of `g⁻¹`. -/
def sigmaPerm (g : G) : Equiv.Perm X :=
  { toFun := sigma g
    invFun := sigma g⁻¹
    left_inv := by
      intro x
      -- Step 1: combine `g⁻¹` and `g` using the action axiom.
      calc
        GroupAction.act g⁻¹ (GroupAction.act g x) =
            GroupAction.act (g⁻¹ * g) x := by
          simpa using (GroupAction.ga_mul g⁻¹ g x).symm
        -- Step 2: simplify `g⁻¹ * g` to `1`.
        _ = GroupAction.act (1 : G) x := by
          -- Alternative: use `congrArg` with `inv_mul_cancel g`.
          -- congrArg (fun t => GroupAction.act t x) (inv_mul_cancel g)
          simp
        -- Step 3: the identity acts as the identity on `X`.
        _ = x := GroupAction.ga_one x
    right_inv := by
      intro x
      -- Step 1: combine `g` and `g⁻¹` using the action axiom.
      calc
        GroupAction.act g (GroupAction.act g⁻¹ x) =
            GroupAction.act (g * g⁻¹) x := by
          simpa using (GroupAction.ga_mul g g⁻¹ x).symm
        -- Step 2: simplify `g * g⁻¹` to `1`.
        _ = GroupAction.act (1 : G) x := by
          -- Alternative: use `congrArg` with `mul_inv_cancel g`.
          -- congrArg (fun t => GroupAction.act t x) (mul_inv_cancel g)
          simp
        -- Step 3: the identity acts as the identity on `X`.
        _ = x := GroupAction.ga_one x }
```

Using `#check` on `sigmaPerm` confirms it has type

```
#check sigmaPerm
```

This confirms the map $g \mapsto \sigma_g$ is a well-defined function from G to $\mathrm{Sym}(X)$. Define the representation and its basic properties:

```
/-- Defines the permutation representation `phi : G → Equiv.Perm X`.
    This sends `g` to the permutation `σ_g`. -/
def phi (g : G) : Equiv.Perm X :=
  sigmaPerm g

/-- `phi` agrees with the action on each element of `X`. -/
lemma phi_apply (g : G) (x : X) : phi g x = GroupAction.act g x := rfl

-- /-- A commented-out attempt at proving multiplicativity of `phi`. -/
-- lemma phi_mul : ∀ (g₁ g₂ : G), phi (g₁ * g₂) = phi g₁ * phi g₂ := by
--   sorry

/-- Shows that `phi` respects multiplication:
    `phi (g₁ * g₂) = phi g₁ * phi g₂`. -/
```

```
lemma phi_mul (g₁ g₂ : G) :
  phi (X := X) (g₁ * g₂) = phi (X := X) g₁ * phi (X := X) g₂ := by
  -- Extensionality reduces equality of permutations to pointwise equality.
  apply Equiv.ext
  intro (x : X)
  calc
    -- Step 1: expand `phi` and the action definition.
    phi (g₁ * g₂) x = GroupAction.act (g₁ * g₂) x := rfl
    -- Step 2: use the action axiom to separate `g₁` and `g₂`.
    _ = GroupAction.act g₁ (GroupAction.act g₂ x) := GroupAction.ga_mul g₁
     ↪  g₂ x

/-- `phi 1` is the identity permutation. -/
lemma phi_one : phi (1 : G) = (1 : Equiv.Perm X) := by
  apply Equiv.ext
  intro (x : X)
  calc
    phi (1 : G) x = GroupAction.act (1 : G) x := rfl
    _ = x := GroupAction.ga_one x
    _ = (1 : Equiv.Perm X) x := by
      simp [Equiv.Perm.one_apply]
```

Finally, package the existence statement:

```
/-- The action yields a permutation representation. -/
theorem group_action_to_perm_representation :
  ∃ (ψ : G → Equiv.Perm X),
    (∀ g x, ψ g x = GroupAction.act g x) ∧
    (∀ g₁ g₂, ψ (g₁ * g₂) = ψ g₁ * ψ g₂) ∧
    (ψ 1 = 1) := by
  exact ⟨phi, ⟨phi_apply, ⟨phi_mul, phi_one⟩⟩⟩
```

5. Stabilizers

For a fixed $x \in X$, the stabilizer is the subset

$$G_x = \{g \in G \mid g \cdot x = x\}.$$

In Lean this is first defined as a set, then shown to be the carrier of a subgroup, and finally packaged as a `Subgroup`.

5.1. Proof sketch (informal). The identity element fixes every x, so $1 \in G_x$. If g_1 and g_2 fix x, then $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = g_1 \cdot x = x$, so $g_1 g_2 \in G_x$. If g fixes x, then $g^{-1} \cdot x = g^{-1} \cdot (g \cdot x) = (g^{-1} g) \cdot x = x$, so $g^{-1} \in G_x$.

5.2. Lean code. Start from the set definition:

```
/-- The stabilizer set
    `G_x = { g ∈ G | g • x = x }`. -/
def stabilizerSet (x : X) : Set G :=
  { g : G | GroupAction.act g x = x }
```

Package it as a subgroup by checking closure: Define the subgroup structure by providing proofs of the subgroup axioms:

- `carrier`: the underlying set is `stabilizerSet x`.
- `one_mem'`: the identity fixes every x by the `ga_one` axiom.
- `mul_mem'`: if g_1 and g_2 fix x, then so does $g_1 g_2$ by the `ga_mul` axiom.
- `inv_mem'`: if g fixes x, then so does g^{-1} by combining `ga_mul` and `ga_one`.

```
/-- The stabilizer `G_x` as a subgroup of `G`. -/
def stabilizer (x : X) : Subgroup G := by
  exact
    { carrier := stabilizerSet (G := G) (X := X) x
      one_mem' := by
        -- The identity fixes every x.
        simp [stabilizerSet, GroupAction.ga_one x]
      mul_mem' := by
        intro g₁ g₂ hg₁ hg₂
        calc
          -- Closure under multiplication via the action axiom.
          GroupAction.act (g₁ * g₂) x = GroupAction.act g₁
          ↪  (GroupAction.act g₂ x) := by
            simpa using (GroupAction.ga_mul g₁ g₂ x)
          _ = GroupAction.act g₁ x := by
            rw [hg₂]
          _ = x := hg₁
      inv_mem' := by
        intro g hg
        calc
          -- If g fixes x, then g⁻¹ fixes x as well.
          GroupAction.act g⁻¹ x = GroupAction.act g⁻¹ (GroupAction.act g
          ↪  x) := by
            rw [hg]
          _ = GroupAction.act (g⁻¹ * g) x := by
            simpa using (GroupAction.ga_mul g⁻¹ g x).symm
          _ = GroupAction.act (1 : G) x := by simp
          _ = x := GroupAction.ga_one x }
```

Then show the set is the carrier of a subgroup:

```
/-- The stabilizer set is the carrier of a subgroup of `G`. -/
theorem stabilizer_set_is_subgroup (x : X) :
    ∃ H : Subgroup G, (H : Set G) = stabilizerSet (G := G) (X := X) x := by
  refine ⟨stabilizer (G := G) (X := X) x, rfl⟩
```

6. What Lean Guarantees

Formalizing these constructions in Lean provides guarantees that informal proofs cannot match. When Lean accepts a definition or theorem, it has verified:

- **Types are correct.** The function `phi : G → Equiv.Perm X` has the claimed type. Lean checks that `sigmaPerm g` is actually a permutation (a bijection), not just a function. If we mistakenly defined a non-bijective map, Lean would reject it.

- **No hidden assumptions.** Every lemma explicitly lists its hypotheses. If a proof uses associativity, the code must invoke `mul_assoc` or the `ga_mul` axiom. There are no "clearly" or "it follows that" steps that skip justification.
- **Axioms match definitions.** The `GroupAction` class requires `ga_mul` and `ga_one`. If we omitted `ga_one`, the proof of `phi_one` would fail because Lean would have no way to show $1 \cdot x = x$.
- **Subgroup structure is verified.** When we package the stabilizer as a `Subgroup`, Lean checks that we provided proofs of closure under multiplication, inverses, and identity. The type system prevents us from claiming "the stabilizer is a subgroup" without proving it.

These checks prevent common errors: mistaken claims about injectivity, forgotten hypotheses, and gaps in subgroup proofs. An external examiner can trust that the Lean code genuinely establishes the mathematical claims, not just that it compiles.

7. Reflection

Writing the proofs in Lean forced me to make every step explicit. In particular:

- I had to show explicitly that $\sigma_{g^{-1}}$ is an inverse.
- I had to unfold the definition of permutation multiplication.

The resulting code is not shorter than a textbook proof, but it is more precise. An external examiner can read the surrounding explanations and then see that each Lean line matches a specific mathematical claim.

7.1. Specific challenges encountered. Several technical hurdles emerged during the formalization:

- **Typeclass resolution:** Ensuring Lean could automatically find the `GroupAction` instance in complex contexts required careful use of explicit type annotations (e.g., `(G := G)`).
- **Equivalence mechanics:** Constructing `sigmaPerm` as an `Equiv.Perm X` required providing both `left_inv` and `right_inv` proofs, which meant carefully applying the `ga_mul` axiom in both directions.
- **Coercions:** Working with subgroups required understanding how Lean coerces elements of type `H` (a subgroup) to type `G` (the ambient group) using `(h : G)`.
- **Function extensionality:** Proving equality of permutations required `Equiv.ext`, and equality of vector-valued functions required `ext i`, which were not immediately obvious.

8. AI Assistance Statement

I used AI-assisted tools in a limited and strictly auxiliary manner. Specifically, AI was used to help automate the conversion of Lean source files into LaTeX code blocks for inclusion in this report, and to assist with reorganising a single Lean file into multiple smaller files during the development process. In addition, AI was occasionally used for high-level brainstorming about structure and presentation.
All mathematical content, Lean definitions, proofs, and formal statements were written, checked, and verified by me. AI tools were not used to replace mathematical reasoning or to generate unverified results. I take full responsibility for the correctness and originality of both the Lean code and the written exposition.

9. References

John B. Fraleigh, Victor J. Katz, *A First Course in Abstract Algebra*, Addison–Wesley, 2003, Section 16 (Group Actions).